SQL Basics - Cheat Sheet

Contents

1. What SQL is

SQL is a declarative, set-based language. You describe what data you want, and the database figures out how to fetch it. That's different from Java where you tell the runtime each step.

"Set-based" matters. A query operates on rows as a group, not one at a time. If you find yourself writing a loop in your head while reading a query, slow down and rethink it.

There's ANSI SQL (the standard) and then there are dialects:

Dialect	Notes
PostgreSQL	Strict, feature-rich, popular for new backends
MySQL	Common, lenient, slight syntax quirks

Dialect	Notes
SQL Server	Microsoft stack, uses TOP instead of LIMIT
Oracle	Enterprise, uses FETCH FIRST N ROWS ONLY
SQLite	Embedded, simple, used in tests and mobile

Most basic SELECT/INSERT/UPDATE/DELETE statements are portable. Pagination, string functions, and date handling are where dialects diverge.

2. Basic SELECT

The shape of every query:

```
SELECT id, name, email
FROM users;
```

Column aliases with AS (the AS is optional in most dialects):

```
SELECT name AS full_name, created_at AS signup_date
FROM users;
```

You can do `SELECT *` to grab every column, but flag this in code reviews. It's fine for ad-hoc exploration. It's risky in production code because adding a column upstream can break consumers.

3. WHERE clause

Filter rows with `WHERE`. The condition must evaluate to true for the row to be returned.

```
SELECT id, name
FROM users
WHERE status = 'active';
```

Comparison operators:

Operator	Meaning
`=`	Equal
`!=` or `<>`	Not equal (both are standard, `<>` is older ANSI)
`<`, `>`, `<=`, `>=`	Standard comparisons
`IN (...)`	Matches any value in a list
`BETWEEN x AND y`	Range, inclusive on both ends

Operator	Meaning
`LIKE 'pattern'`	String pattern matching
`IS NULL / IS NOT NULL`	NULL check

Examples:

```
SELECT * FROM orders WHERE total > 100;
SELECT * FROM users WHERE status IN ('active', 'pending');
SELECT * FROM orders WHERE created_at BETWEEN '2025-01-01' AND '2025-01-31';
SELECT * FROM users WHERE email LIKE '%@gmail.com';
SELECT * FROM users WHERE deleted_at IS NULL;
```

LIKE wildcards: `%` matches any number of characters (including zero). `_` matches exactly one character. So `'A%'` finds anything starting with A. `'A_'` finds two-character strings starting with A.

4. Boolean logic

Combine conditions with `AND`, `OR`, `NOT`. Parenthesize when mixing them so you don't get surprised.

```
SELECT * FROM orders
WHERE status = 'pending'
  AND (total > 500 OR user_id IN (1, 2, 3));
```

Without parens, `AND` binds tighter than `OR`, which trips people up. When in doubt, add parens. Readers will thank you.

5. ORDER BY

Sort the result. Default is `ASC`.

```
SELECT id, name, created_at
FROM users
ORDER BY created_at DESC;
```

Multiple columns:

```
SELECT * FROM orders
ORDER BY status ASC, created_at DESC;
```

NULL ordering varies by dialect. Postgres puts NULLs last by default for ASC. You can force it:

```
ORDER BY created_at DESC NULLS LAST;
```

MySQL doesn't support NULLS FIRST/LAST directly. You'd do `ORDER BY created_at IS NULL, created_at DESC` instead.

6. LIMIT and OFFSET

Paginate results. Postgres and MySQL:

```
SELECT * FROM users
ORDER BY id
LIMIT 20 OFFSET 40;
```

That gives you rows 41 to 60. Always pair `LIMIT` / `OFFSET` with `ORDER BY`. Without an order, the database can return rows in any order, and your pagination breaks.

Oracle and modern SQL Server use ANSI syntax:

```
SELECT * FROM users
ORDER BY id
OFFSET 40 ROWS
FETCH FIRST 20 ROWS ONLY;
```

For large offsets, performance gets bad (the database still scans the skipped rows). Keyset pagination handles that better, covered in the intermediate sheet.

7. DISTINCT

Strips duplicate rows from the result set.

```
SELECT DISTINCT status FROM orders;
```

DISTINCT applies to the whole row, not just one column. So `SELECT DISTINCT status, user_id` gives you unique combinations of both.

It isn't free. The database has to sort or hash to deduplicate. If you're using it as a "fix" for a bad join, fix the join instead.

8. INSERT

Single row:

```
INSERT INTO users (name, email, status)
VALUES ('Alice', 'alice@example.com', 'active');
```

Multi-row:

```sql
INSERT INTO users (name, email, status)
VALUES
  ('Bob', 'bob@example.com', 'active'),
  ('Carol', 'carol@example.com', 'pending');
```

Insert from another query:

```sql
INSERT INTO archived_users (id, name, email)
SELECT id, name, email
FROM users
WHERE status = 'deleted';
```

If you skip the column list, you must supply values for every column in table order. Skipping the column list is brittle, always name your columns.

9. UPDATE

Modify existing rows. Always include a `WHERE` clause unless you really mean to update every row.

```sql
UPDATE users
SET status = 'inactive'
WHERE last_login < '2024-01-01';
```

You can update multiple columns at once:

```sql
UPDATE orders
SET status = 'shipped', shipped_at = NOW()
WHERE id = 12345;
```

`UPDATE users SET status = 'inactive';` with no `WHERE` updates every row in the table. Some teams require `BEGIN; ... ROLLBACK;` testing first to avoid this kind of accident.

10. DELETE

Same shape as `UPDATE`, same warning.

```sql
DELETE FROM users WHERE id = 42;
```

`DELETE FROM users;` (no `WHERE`) wipes every row. The table structure stays.

`TRUNCATE TABLE users;` is the fast alternative for emptying a table. It skips per-row logging, can't be filtered with `WHERE`, and usually can't be rolled back. Use it for test data resets, not for production with foreign keys hanging off.

11. Basic JOINs

Joins combine rows from two tables based on a matching column. Pretend we have:

```
users(id, name, email, created_at, status)
orders(id, user_id, total, status, created_at)
```

`INNER JOIN` returns rows where both sides match:

```
SELECT u.name, o.total
FROM users u
INNER JOIN orders o ON o.user_id = u.id;
```

If a user has no orders, they're excluded. If an order has no matching user, it's excluded.

`LEFT JOIN` keeps every row from the left table, even if the right side has no match. Missing columns come back as NULL.

```
SELECT u.name, o.total
FROM users u
LEFT JOIN orders o ON o.user_id = u.id;
```

That gives you every user, with NULL totals for users who never ordered. Useful for "find users without orders":

```
SELECT u.id, u.name
FROM users u
LEFT JOIN orders o ON o.user_id = u.id
WHERE o.id IS NULL;
```

`RIGHT JOIN` is the mirror image. Rarely used in practice, most people just flip the table order and use `LEFT JOIN`.

`USING(col)` is shorter syntax when both tables share the join column name:

```
SELECT u.name, o.total
FROM users u
JOIN orders o USING (user_id);
```

But `users` has `id`, not `user_id`, so this wouldn't work for our schema. `ON` is more flexible because the column names can differ.

12. NULL handling

NULL is the absence of a value. It means "we don't know". It's never equal to zero or an empty string, and any comparison with it returns unknown.

SQL uses three-valued logic: true, false, unknown.

```
SELECT NULL = NULL;         -- returns NULL (unknown), not true
SELECT NULL != 'something'; -- returns NULL
SELECT NULL = 0;            -- returns NULL
```

So this query returns zero rows:

```
SELECT * FROM users WHERE deleted_at = NULL;          -- wrong
```

You need:

```
SELECT * FROM users WHERE deleted_at IS NULL;         -- right
```

NULLs in aggregates: `COUNT(*)` counts all rows. `COUNT(col)` ignores rows where col is NULL. `SUM`, `AVG`, `MIN`, `MAX` skip NULLs. So `AVG(price)` on five rows where one is NULL averages four values, not five.

13. Aggregate functions

Functions that collapse many rows into one value.

Function	Behavior
`COUNT(*)`	Count all rows
`COUNT(col)`	Count rows where col is not NULL
`COUNT(DISTINCT col)`	Count unique non-NULL values
`SUM(col)`	Sum, ignores NULLs
`AVG(col)`	Average, ignores NULLs
`MIN(col)`	Smallest value
`MAX(col)`	Largest value

```sql
SELECT
  COUNT(*)   AS total_orders,
  SUM(total) AS revenue,
  AVG(total) AS avg_order_value,
  MIN(total) AS smallest,
  MAX(total) AS largest
FROM orders
WHERE status = 'completed';
```

The difference between `COUNT(*)` and `COUNT(col)` matters when the column is nullable. `COUNT(email)` skips users with no email.

14. GROUP BY

Aggregate per group instead of across the whole table.

```sql
SELECT user_id, COUNT(*) AS order_count, SUM(total) AS spent
FROM orders
GROUP BY user_id;
```

> **Rule**
>
> every column in the SELECT list that isn't inside an aggregate must appear in `GROUP BY`. Postgres and Oracle enforce this strictly. MySQL is looser in older versions and lets you violate it, with unpredictable results.

```sql
-- Wrong: name is selected but not grouped or aggregated
SELECT user_id, name, COUNT(*)
FROM orders
GROUP BY user_id;
```

Fix by either grouping by name too or wrapping it in an aggregate like `MIN(name)`. Or, more usefully, join to users and group by `users.id`:

```sql
SELECT u.id, u.name, COUNT(o.id) AS order_count
FROM users u
LEFT JOIN orders o ON o.user_id = u.id
GROUP BY u.id, u.name;
```

15. HAVING

Filter groups after aggregation. `WHERE` filters rows before grouping. `HAVING` filters groups after.

```sql
SELECT user_id, COUNT(*) AS order_count
FROM orders
WHERE status = 'completed'   -- filters individual orders
GROUP BY user_id
HAVING COUNT(*) > 5;         -- filters resulting groups
```

`WHERE` can't reference aggregates because the aggregation hasn't happened yet. That's why `HAVING` exists.

```sql
-- Wrong
SELECT user_id, COUNT(*) FROM orders
WHERE COUNT(*) > 5 GROUP BY user_id;

-- Right
SELECT user_id, COUNT(*) FROM orders
GROUP BY user_id HAVING COUNT(*) > 5;
```

16. CASE expressions

SQL's version of if/else. Two forms.

Simple form, like a switch:

```sql
SELECT id,
  CASE status
    WHEN 'active' THEN 'A'
    WHEN 'pending' THEN 'P'
    ELSE 'X'
  END AS status_code
FROM users;
```

Searched form, full conditions:

```sql
SELECT id, total,
  CASE
    WHEN total < 50 THEN 'small'
    WHEN total < 500 THEN 'medium'
    ELSE 'large'
  END AS bucket
FROM orders;
```

CASE works anywhere an expression works: SELECT, WHERE, ORDER BY, even GROUP BY.

```sql
SELECT
  CASE WHEN total < 100 THEN 'low' ELSE 'high' END AS tier,
  COUNT(*) AS cnt
FROM orders
GROUP BY CASE WHEN total < 100 THEN 'low' ELSE 'high' END;
```

17. Common data types

A quick map. Exact names vary by dialect.

Category	Common types
Integer	`INT`, `BIGINT`, `SMALLINT`
Decimal	`DECIMAL(p, s)` (exact), `NUMERIC` (alias)
Float	`REAL`, `DOUBLE PRECISION` (avoid for money)
String	`VARCHAR(n)`, `TEXT`, `CHAR(n)` (fixed length, rarely used)
Date/time	`DATE`, `TIME`, `TIMESTAMP`, `TIMESTAMPTZ` (with time zone)
Boolean	`BOOLEAN` (Postgres), `TINYINT(1)` (MySQL)
JSON	`JSON`, `JSONB` (Postgres, indexed binary form)

Don't store money in floats. 0.1 + 0.2 isn't 0.3 in IEEE 754. Use `DECIMAL(19, 4)` or similar.

For timestamps, prefer the time-zone-aware type when the dialect offers one. Stripping zones causes pain later.

18. Conditional helpers

`COALESCE` returns the first non-NULL argument.

```sql
SELECT name, COALESCE(nickname, name) AS display_name
FROM users;
```

Useful for default values when joining or when a column might be NULL. The intermediate sheet covers `NULLIF`, `GREATEST`, `LEAST`, and string/date functions in more detail.

19. Best practices

- Always name your columns in INSERT, don't rely on table order.
- Always include `WHERE` in UPDATE and DELETE unless you really mean every row.

- Always pair `LIMIT` with `ORDER BY` for predictable pagination.
- Use `IS NULL` / `IS NOT NULL`, never `= NULL`.
- Parenthesize mixed AND/OR conditions.
- Avoid `SELECT *` in production queries. Name the columns you need.
- Use table aliases (`users u`, `orders o`) when joining, but keep them short and consistent.
- Format multi-line queries with one clause per line. Future you will read this code more than you write it.

20. Common gotchas

- `NULL = NULL` is unknown, not true. Same for `NULL != 'x'`. Always use `IS NULL`.
- `COUNT(col)` skips NULLs. `COUNT(*)` doesn't. If the count's off, check for nullable columns.
- MySQL allows non-grouped columns in SELECT with GROUP BY. The values are undefined. Postgres rejects this outright.
- `LIKE 'abc%'` can use an index. `LIKE '%abc'` (leading wildcard) usually can't.
- `BETWEEN` is inclusive on both ends. Date ranges with timestamps are easy to mess up: `BETWEEN '2025-01-01' AND '2025-01-31'` misses anything on Jan 31 after midnight.
- TRUNCATE isn't transactional in some dialects. You can't roll it back.
- OR conditions across different columns often defeat index use. Watch query plans.
- Floats for money. Don't.
- Big OFFSET values get slow because the database still walks the skipped rows.
- Mixing `INNER JOIN` and `LEFT JOIN` with `WHERE` clauses on the right table silently turns the `LEFT JOIN` into an `INNER JOIN`. Put the right-side filter in the ON clause if you want to keep the outer behavior.

21. Quick reference

Task	Syntax
Select with filter	`SELECT col FROM tbl WHERE cond;`
Multiple conditions	`WHERE a = 1 AND (b = 2 OR c = 3)`
Pattern match	`WHERE name LIKE 'A%'`
NULL check	`WHERE col IS NULL`
Sort	`ORDER BY col DESC`
Paginate (PG/MySQL)	`LIMIT 20 OFFSET 40`
Paginate (Oracle/MSSQL)	`OFFSET 40 ROWS FETCH FIRST 20 ROWS ONLY`
Dedupe	`SELECT DISTINCT col FROM tbl`
Insert one	`INSERT INTO tbl (a, b) VALUES (1, 2);`
Insert many	`INSERT INTO tbl (a, b) VALUES (1,2), (3,4);`

Task	Syntax
Update	`UPDATE tbl SET a = 1 WHERE id = 5;`
Delete	`DELETE FROM tbl WHERE id = 5;`
Empty table fast	`TRUNCATE TABLE tbl;`
Inner join	`FROM a JOIN b ON a.id = b.a_id`
Outer join	`FROM a LEFT JOIN b ON a.id = b.a_id`
Count rows	`SELECT COUNT(*) FROM tbl;`
Group counts	`GROUP BY col`
Filter groups	`HAVING COUNT(*) > 5`
If/else	`CASE WHEN x THEN y ELSE z END`
Default for NULL	`COALESCE(a, b, 'default')`